

Ejercicios de Análisis de Algoritmos

Jaime C Smith

07/30/2026

1. Analyze the bubble_sort algorithm using Big-O notation.

Answer

- Time complexity (worst case): $O(n^2)$

Why?

- Let n be the number of elements in the list.
- bubble_sort uses two loops, one inside the other.
- The outer loop runs n times.
- For each outer loop, the inner loop also runs about n times.
- So, the total work is roughly $n \times n = n^2$.
- That gives us $O(n^2)$: time grows like the square of the list size.

2. Analyze print_numbers_times_2 using Big-O notation.

I. print_numbers_times_2

```
def print_numbers_times_2(numbers_list):  
    for number in numbers_list:  
        print(number * 2)
```

Answer

- Time complexity (worst case): $O(n)$

Why?

- Let n be the length of numbers_list.
- There is one loop that goes through all elements once.
- Each step does a simple calculation and a print (both constant time).

- Total work is proportional to n , so the complexity is $O(n)$.

II. check_if_lists_have_an_equal

```
def check_if_lists_have_an_equal(list_a, list_b):
    for element_a in list_a:
        for element_b in list_b:
            if element_a == element_b:
                return True
    return False
```

Answer

- Time complexity (worst case): $O(n \cdot m)$
 - If both lists have similar size, this is often written as $O(n^2)$.

Why?

- Let n be the length of list_a, and m the length of list_b.
- For **each** element in list_a, we check **all** elements in list_b.
- In the worst case, there is no match, so we do all comparisons:
 - about $n \times m$ checks.
- That gives us $O(n \cdot m)$.
- If both lists grow together and are about the same size, that looks like $O(n^2)$.

III. print_10_or_less_elements

```
def print_10_or_less_elements(list_to_print):
    list_len = len(list_to_print)
    for index in range(min(list_len, 10)):
        print(list_to_print[index])
```

Answer

- Time complexity (worst case): $O(1)$ (constant time)

Why?

- The loop runs at **most 10 times**, no matter how long the list is.
- A fixed number of steps (like 10) does **not** grow with the input size.
- That's what constant time means: $O(1)$.

IV. generate_list_trios

```
def generate_list_trios(list_a, list_b, list_c):
    result_list = []
    for element_a in list_a:
        for element_b in list_b:
            for element_c in list_c:
                result_list.append(f'{element_a} {element_b} {element_c}')
    return result_list
```

Answer

- **Time complexity (worst case):**
 - $O(a \cdot b \cdot c)$ in terms of the three list sizes
 - If all three lists are about the same size, this is $O(n^3)$.

Why?

- Let:
 - a = length of list_a
 - b = length of list_b
 - c = length of list_c
- There are **three nested loops**:
 - For each element in list_a, we loop over all of list_b.
 - For each pair from list_a and list_b, we loop over all of list_c.
- Total combinations: $a \times b \times c$.
- The work grows with the product of the three sizes $\rightarrow O(a \cdot b \cdot c)$.
- If the three lists are similar in size ($a \approx b \approx c \approx n$), this becomes $O(n^3)$.